

Football Tournament Manager

Task 1 Report · Task 2 Report · Final Project Report

Professor: Pietro Ferrara

Student: Mohammadreza Haji Tayebi

Student Number: 908462

GitHub: <https://github.com/samantayebi/football-tournament>

Live: <http://tournament.samyland.com>

Contents

Task Report 1 — The Kata

Task Report 2 — Architecture Characteristics

Final Project Report — Architecture and Implementation

Task Report 1: The Kata

1. Task Objective

Task 1 asked us to write a kata — a short domain-level description of the IT system we wanted to build. The kata should describe the problem, who the users are, what the system needs to do, and any context that affects the design later. The point was to have a foundation for identifying architecture characteristics in Task 2.

2. Understanding the Task

A kata is not a technical document. It is more like a brief from a client: here is the problem, here are the users, here is what we need. At this stage you are not supposed to think about databases or services. You are just trying to understand the domain well enough to make good architecture decisions later.

We chose football. Specifically, local amateur single-elimination tournaments. An earlier draft of the kata included live in-match scoring and referee management, but we removed those because they would have made the architecture more complex without adding real value to what the system needs to do.

3. The Kata

Description

The Football Tournament Manager is a web platform for organizing single-elimination football tournaments for local amateur clubs. It covers the full lifecycle: teams enroll, the organizer generates a bracket, matches are played, the organizer enters results, and the bracket advances automatically. The public can follow the tournament online without logging in. The system supports up to 32 teams and multiple tournaments.

Users

There are three types of user. The tournament organizer (1 to 3 per tournament, non-technical) manages enrollment, the bracket, and results through an admin panel. Public visitors (up to around 200 per tournament) view the bracket, standings, and match reports without logging in. Team representatives (one per team, up to 32) submit their enrollment and view their team's information.

Requirements

- R1: Teams can submit their name, contact email, and player roster to enroll in a tournament.
- R2: The organizer reviews enrollment requests and approves or rejects each team.
- R3: The system generates a single-elimination bracket from approved teams. The draw can be random or seeded.
- R4: The organizer can assign a date, time, and venue to each match.
- R5: The organizer enters the final score. The system determines the winner and advances them to the next round.
- R6: The current state of the bracket is visible to all visitors without logging in.
- R7: The system auto-generates a match report when a result is entered. The organizer can edit it.
- R8: The system shows standings: wins, losses, goals scored, goals conceded.
- R9: The organizer can create, switch between, and archive multiple tournaments.
- R10: Individual player goals are tracked per match. A top scorers leaderboard is available.

Additional Context

- The organizer is not technical. The interface has to be simple and work without instructions.
- Traffic is low and unpredictable: mostly idle, with short spikes on match days.
- The system runs on a single VPS (IONOS, Ubuntu 24.04).
- There is one developer working alone on a limited budget.
- The public should be able to follow results without installing an app.
- Results are entered after each match ends, not during it.

4. Decisions Made at This Stage

Restricting the scope to single-elimination format only was the most important decision here. There are no group stages, no round-robin calculations, no tie-breaking rules. A match has exactly one winner. This made the bracket logic simple enough to be a pure function that is easy to unit test later.

Setting the maximum at 32 teams also fed directly into Task 2. It told us that the system does not need to handle thousands of teams or large-scale concurrent updates. The scalability target in Task 2 — 50 concurrent users with p95 under 500ms — came directly from this context.

5. Result

The kata gave us 10 numbered requirements (R1 to R10) that we referred to throughout the rest of the project. Every implemented feature traces back to one of them. Nothing was built that is not in the kata.

6. Difficulties

The hardest part was deciding what to leave out. The first version of the kata was too large. Cutting it down meant accepting that some things (referee management, group chats between teams) would not be built. This was the right decision — a smaller scope with a cleaner architecture is better than a large scope with a messy one.

7. Connection to the Rest of the Project

Each requirement maps directly to a module in the final system. R1 and R2 are in `monolith/modules/enrollment/`. R3, R4, and R5 are in `monolith/modules/bracket/` and `monolith/modules/results/`. R6 is served by `public-service`. R7 is in `monolith/modules/reports/`. R8 is in `standings-service`. R9 is in `monolith/modules/tournament/`. R10 is in `monolith/modules/stats/`.

8. Conclusion

Writing the kata before anything else was useful. It forced us to think about who would actually use the system and what they needed, rather than jumping straight to technology choices. The narrow scope made every subsequent decision easier.

Task Report 2: Architecture Characteristics

1. Task Objective

Task 2 asked us to identify which architecture characteristics are relevant to our system, connect each one to specific requirements from the kata, and give a measurable target for each. The professor was clear: do not try to satisfy every characteristic. Each one adds complexity, so only include the ones that are genuinely necessary.

2. Understanding the Task

Architecture characteristics are basically the non-functional requirements: how fast does the system need to be, how easy to deploy, how well does it handle failures. The point is to decide these before writing any code, so that architecture decisions later have a clear reason behind them.

We applied the three criteria from Lecture 2 to filter the list: the characteristic must be non-domain, it must influence the structural design, and it must be critical for the system to succeed.

3. The Nine Characteristics

We identified nine characteristics. For each one we describe which kata requirement led to it, the quantified target we set, and what we actually measured in the end.

Simplicity

Priority: High. Kata reference: All requirements and context.

Why we chose it: The organizer is non-technical. A system that is hard to understand will not be maintained. This characteristic influenced the choice of domain partitioning over layered architecture.

Target: Maximum 2 architectural layers. A new developer should understand the system in one day.

What we measured: 2 layers: the monolith (write side) and three services (read side). 7 named domain modules. Verified manually.

Maintainability

Priority: High. Kata reference: R5, R7, R8.

Why we chose it: One developer, many features to add. If a change to one feature breaks another, development gets slow.

Target: A single feature change touches at most 1 module on the backend and 1 component on the frontend.

What we measured: Verified across 15 feature additions during the expansion phase. Each one touched 1 to 2 modules at most.

Deployability

Priority: High. Kata reference: Context.

Why we chose it: Single VPS, non-technical operator. The system has to start with one command and work the same way every time.

Target: Full system starts with docker compose up --build. No manual steps after setting up the .env file.

What we measured: Verified on macOS and Ubuntu 24.04. All 11 containers start in the correct order.

Modularity

Priority: Medium. Kata reference: R1 to R10.

Why we chose it: The system has 7 different domains. If the code is tangled, any change becomes risky.

Target: Zero cyclic dependencies. Each module exposes one public index.js.

What we measured: ESLint passes on all modules. Verified with npm test.

Testability

Priority: Medium. Kata reference: R3, R5, R8.

Why we chose it: The bracket algorithm and winner determination are the most critical pieces of logic. A wrong result would be visible to everyone.

Target: At least 60% line coverage on core business logic modules.

What we measured: 100% line coverage on bracketUtils.js, resultUtils.js, standingsUtils.js. 12 out of 12 tests pass.

Scalability

Priority: Medium. Kata reference: R6, R9.

Why we chose it: The public bracket page gets traffic from all visitors at once on match day.

Target: 50 concurrent users, p95 response time under 500ms on public endpoints.

What we measured: k6 load test: p95 = 16.27ms, 0% errors, 62 requests per second at peak. All 4 thresholds passed.

Fault tolerance

Priority: Low. Kata reference: Context, course requirement.

Why we chose it: Three distributed services are required by the course. If one crashes, the organizer must still be able to enter results.

Target: The monolith stays responsive if any service crashes. Services auto-restart within 30 seconds.

What we measured: Docker restart:unless-stopped on all services. Health endpoints on all 4 services return status ok.

Observability

Priority: Medium. Kata reference: All services.

Why we chose it: We added three distributed services. Without monitoring it is hard to know what is happening inside the system.

Target: All backend services expose a /metrics endpoint. Grafana shows live data.

What we measured: Prometheus scrapes 3 services every 15 seconds. 10-panel Grafana dashboard loads automatically at startup.

Security

Priority: Medium. Kata reference: Context.

Why we chose it: The system is public-facing. Admin endpoints need authentication. Public endpoints need protection against abusive traffic.

Target: JWT authentication on all admin routes. Nginx rate limiting on all routes.

What we measured: Nginx returns 429 when the rate limit is exceeded. All admin routes return 401 without a valid token.

4. What We Left Out and Why

Several characteristics were considered and rejected. Elasticity was removed because the context specifies low predictable traffic — around 200 visitors. Elasticity requires cloud autoscaling, which is unnecessary at this scale. Performance as a separate characteristic was removed because scalability already covers it — p95 under 500ms at 50 users is a performance target. Reliability as a separate characteristic was removed because fault tolerance covers what we actually need: surviving a service crash. Full high-availability reliability (database replication, zero downtime) is out of scope for a single VPS. Overall cost was also removed — everything is open-source on a cheap VPS and is not a structural concern.

5. Result

We ended up with 9 characteristics, each with a concrete target. All 9 were verified in the final implementation. The most measurable result was scalability: the target was p95 under 500ms, and the actual result was 16ms — about 30 times better than required. The main reason for this is the Redis cache in public-service, which serves the bracket and standings from memory for most requests.

6. Difficulties

Quantifying characteristics like simplicity and maintainability was hard. You cannot write a test that checks whether a developer understands the system in one day. We verified these manually by tracking which files changed during each feature addition. This is honest but not perfect — automated checks for structural characteristics would be better.

7. Connection to the Rest of the Project

Every technology choice in Task 3 traces back to one of these characteristics. Docker Compose came from deployability. Domain partitioning came from maintainability and modularity. Redis came from scalability. RabbitMQ came from fault tolerance. JWT and rate limiting came from security. Having these targets written down before building anything made every decision easier to justify.

8. Conclusion

Task 2 was probably the most useful of the three tasks. Deciding what the system needs to be good at before writing any code changed how we made every subsequent decision. Without this, we would have picked technologies out of habit rather than because they solved a specific problem.

Final Project Report

Football Tournament Manager — Architecture and Implementation

1. Introduction

This report describes the architecture and implementation of the Football Tournament Manager, a web system built as the final project for the Software Architectures course at Ca' Foscari University of Venice. The system manages the full lifecycle of single-elimination football tournaments: teams enroll, the organizer manages the bracket, results are entered, and the public follows the tournament online.

We built the system in two phases. The first phase covered the basic kata requirements: enrollment, bracket, results, standings, reports. The second phase added the architectural depth: event-driven communication via RabbitMQ, a Redis cache, Prometheus and Grafana monitoring, Server-Sent Events for real-time updates in the browser, and a set of extra domain features. The final system runs 11 Docker containers and is deployed at tournament.samyland.com.

We used Claude Code (by Anthropic) for implementation. The architecture was designed before any code was written, and every prompt to the AI was based on that design. This is discussed in detail in Section 13.

2. Scope and Requirements

The system targets single-elimination football tournaments with up to 32 teams. The main users are the tournament organizer (non-technical, uses an admin panel) and public visitors (read-only access, no login needed).

All 10 kata requirements were implemented:

- R1: Team enrollment with name, contact email, and player roster.
- R2: Organizer approves or rejects enrollment requests.
- R3: Single-elimination bracket generation — random or seeded.
- R4: Match scheduling: date, time, venue.
- R5: Score entry, automatic winner determination, bracket advancement.
- R6: Public bracket viewer — no login required, updates in real time.
- R7: Match report — auto-generated when a result is entered, editable by organizer.
- R8: Team standings: wins, losses, goals scored, goals conceded.
- R9: Multiple tournaments — create, switch between, archive.
- R10: Player goal tracking per match and top scorers leaderboard.

Beyond the kata we also implemented: bracket seeding, team logo upload, live match commentary, a tournament history page, dark and light mode, and Swagger API documentation.

3. Architecture Overview

We chose a hybrid architecture that combines three styles. This was not arbitrary — each style was chosen because of a specific characteristic from Task 2.

The write side uses a modular monolith. All admin operations — enrolling teams, generating the bracket, entering results, writing reports — need to happen inside the same database transaction. If we split them into separate services, we would need distributed transactions, which are complex and slow. A monolith keeps this simple. Inside the monolith, the code is

organized by business domain rather than by technical layer. This means a change to the commentary feature only touches monolith/modules/commentary/, not multiple layers spread across the codebase.

The read side uses service-based architecture. Standings and public bracket data are served by separate services that share the same PostgreSQL database. We chose service-based (shared database) over microservices (separate databases) because the data is too connected. Standings queries join teams and matches. Separating those into different databases would require data synchronization across services. The professor's Lecture 4 describes service-based as the right choice when services share data heavily.

Asynchronous communication between the monolith and the services uses RabbitMQ. When a result is entered, three things need to happen: the Redis cache is cleared, standings-service logs a trigger, and notification-service logs the event. Rather than calling each one directly and waiting for them, the monolith publishes one message to RabbitMQ and returns. Each consumer handles it in the background. This means adding a new consumer later requires zero changes to the monolith.

This combination sits between a pure monolith and full microservices. It takes the simplicity of a shared database and the decoupling of asynchronous events, without the overhead of distributed transactions and data isolation.

4. System Components

The system has 11 Docker containers. Here is what each one does:

- monolith (monolith/): The write side. 7 domain modules, JWT authentication, RabbitMQevent publishing, Prometheus metrics, Winston logging.
- standings-service (standings-service/index.js): Computes and serves team statistics. Consumes RabbitMQ events. Has /metrics and /health endpoints.
- public-service (public-service/index.js): Serves public bracket and standings with Redis cache. Manages SSE connections for browser updates. Has /metrics and /health.
- notification-service (notification-service/index.js): Consumes RabbitMQ events and logsthem. Designed as an extension point for future email or push notifications.
- gateway (Nginx) (gateway/nginx.conf): Routes all /api/v1/* traffic to the right service. Handles rate limiting, SSE proxy settings, and serves frontend static files.
- db (PostgreSQL 16) (monolith/db/schema.sql): One shared database for all backendservices. 8 tables.
- rabbitmq (docker-compose.yml): Message broker. One fanout exchange calledtournament_events.
- redis (public-service/index.js): Cache for bracket and standings. Invalidatedautomatically when a result is entered.

- prometheus (monitoring/prometheus.yml): Scrapes /metrics from 3 services every 15 seconds.
- grafana (monitoring/grafana-dashboard.json): 10-panel live dashboard. Loads automatically at startup via a provisioning file.
- frontend (frontend/src/): React 18 single-page app. Built to static files and served by Nginx.

The 7 domain modules inside the monolith:

- enrollment (monolith/modules/enrollment/): Team registration, approval, player roster, seeding, logo upload.
- bracket (monolith/modules/bracket/): Bracket generation (random or seeded), match scheduling.
- results (monolith/modules/results/): Score entry, winner determination, bracket advancement, auto-report trigger.
- reports (monolith/modules/reports/): Match report creation and update using ONCONFLICT upsert.
- tournament (monolith/modules/tournament/): Tournament CRUD, stats aggregation, archive.
- stats (monolith/modules/stats/): Goal recording via the match_goals table, top scorers query.
- commentary (monolith/modules/commentary/): Live text commentary per match via the match_commentary table.

5. Data Design

All data lives in one PostgreSQL 16 database shared by the monolith, standings-service, and public-service. The schema is defined in monolith/db/schema.sql. We have 8 tables.

- tournament: Root table. Columns: id, name, format (always single-elimination), status (enrollment or completed).
- teams: One row per team per tournament. Columns include tournament_id, status, seed (for ordered bracket generation), and logo_url (path to uploaded image).
- players: One row per player. Linked to a team. Used for goal attribution in match_goals.
- matches: The bracket structure. Each row is one match with tournament_id, roundnumber, team1_id, team2_id, scores, winner_id, and status. A NULL team ID means that slot is still TBD.
- reports: One report per match. Has a UNIQUE constraint on match_id, which allows the ON CONFLICT upsert pattern. Reports are auto-generated when a result is entered.
- match_goals: One row per goal. Stores match_id, player_id, team_id, and minute (optional).

- match_commentary: One row per commentary entry. Has a type field: goal,yellow_card, red_card, substitution, info, or comment.

One design choice worth explaining: tournament_id is a foreign key on both teams and matches, not just one of them. This means every query is scoped to a specific tournament by default. It is what made the multiple tournaments feature straightforward to add — we did not need to rework the data model at all. The Redis cache keys also include the tournament ID: bracket:1, standings:2, and so on.

6. API and Communication

All routes are versioned under /api/v1/ and go through the Nginx gateway. The frontend talks to the backend via axios using a relative base URL (frontend/src/api.js). This was important for the VPS deployment — using http://localhost as the base URL would fail in production because the browser connects to tournament.samyland.com, not localhost. We discovered this bug after deploying and fixed it by changing the base URL to an empty string.

The main routes are:

- POST /api/v1/admin/login (auth: No) — Returns a JWT token.
- GET/POST/PATCH /api/v1/admin/enrollment/* (auth: JWT) — Team registration, approval, seed assignment, logo upload.
- GET/POST/PATCH /api/v1/admin/bracket/* (auth: JWT) — Generate bracket, view bracket, schedule a match.
- PATCH /api/v1/admin/matches/:id/result (auth: JWT) — Enter a result. This triggers the RabbitMQ event.
- POST/GET /api/v1/admin/matches/:id/goals (auth: JWT) — Record and retrieve goalscorers for a match.
- POST/GET /api/v1/admin/matches/:id/commentary (auth: JWT (POST only)) — Add and read commentary.
- GET/POST/PATCH /api/v1/admin/tournaments/* (auth: JWT) — Tournament management.
- GET /api/v1/public/bracket (auth: No) — Bracket data, served from Redis cache when available.
- GET /api/v1/public/standings (auth: No) — Standings, served from Redis cache when available.
- GET /api/v1/public/events (auth: No) — SSE stream — browser subscribes to get live update notifications.
- GET /api/v1/public/tournaments (auth: No) — List of all tournaments, used to populate the navbar dropdown.
- GET /api/v1/public/top-scorers (auth: No) — Top scorers leaderboard.

- GET /api/v1/public/reports/:matchId (auth: No) — Match report, publicly accessible.
- GET /api/v1/public/matches/:matchId/commentary (auth: No) — Commentary timeline for a match.
- GET /health/* (auth: No) — Health check per service, returns JSON with dependency status.
- GET /metrics (auth: No) — Prometheus metrics endpoint, on monolith, standings, and public.
- GET /api-docs (auth: No) — Interactive Swagger API documentation.

The real-time update chain: the admin enters a result → the monolith saves it to PostgreSQL and publishes match.completed to RabbitMQ → public-service receives the event, deletes the cache entries for bracket and standings, and sends an SSE message to all connected browser tabs → each tab re-fetches the bracket and standings automatically.

The whole chain takes under 100 milliseconds.

7. Key Architecture Decisions

This section explains the five most important decisions, why we made them, and what the trade-offs were.

Shared database instead of separate databases

We chose to have all backend services share one PostgreSQL database rather than giving each service its own. The main reason: a result entry needs to update the match, advance the winner to the next round, and generate a report all in one database transaction. If these were spread across separate databases, handling a failure halfway through would require distributed transactions, which are complex and slow. One shared database makes this a simple atomic operation.

The trade-off is that all services are affected if the database has a problem. We also cannot use different database technologies per service. For the scale of this project, this is acceptable.

RabbitMQ fanout exchange for async communication

When a result is entered, three things need to happen downstream: the Redis cache must be cleared, standings-service needs to know, and notification-service should log the event. We chose to use RabbitMQ rather than calling each service directly. The reason: if we called them one by one, the result entry would block until all three responded. With RabbitMQ, the monolith publishes one message and returns immediately. The consumers handle it asynchronously.

This also means adding a new consumer — for example, sending an email notification — requires zero changes to the monolith. The trade-off is operational complexity: RabbitMQ is one more container to manage, and if it is unavailable, events are lost rather than retried.

Redis read-through cache with event-driven invalidation

The public bracket and standings pages are read by every visitor, but they only change when a result is entered. Without caching, every page view hits PostgreSQL. With Redis, the first request fetches from the database and stores the result in memory. Subsequent requests are served in under 1ms. When a match.completed event arrives from RabbitMQ, the cache entries are deleted immediately, so the next request always gets current data. There is also a 60-second TTL as a safety net.

The trade-off is that cache invalidation adds complexity. It requires the public-service to also be connected to RabbitMQ. If Redis is rebuilt, the cache is empty and the first request per tournament is slightly slower until it fills again.

Server-Sent Events instead of WebSockets

We needed the bracket page to update automatically when a result is entered. We chose SSE over WebSockets because the update is strictly one-directional: the server tells the browser something changed, and the browser re-fetches. SSE uses a standard HTTP connection, works through Nginx without special configuration, and has native browser support without any library. WebSockets would have added complexity for no extra benefit in our case.

One limitation: SSE cannot be used for browser-to-server streaming if that becomes a requirement later. For now it covers everything we need.

Domain partitioning inside the monolith

The code inside the monolith is organized by business domain (enrollment, bracket, results, etc.) rather than by technical layer (controllers, services, repositories). We chose this because changes in this system tend to be domain-specific. If we add a field to commentary, we touch monolith/modules/commentary/ and nothing else. With a layered architecture that same change would spread across multiple files in different folders.

This was verified during the expansion phase: all 15 new features touched at most 2 domain modules. This makes the project easier to maintain, especially when working alone.

8. Architecture Characteristics

This section connects each characteristic from Task 2 to the actual implementation.

Simplicity

Target: Max 2 architectural layers. Understandable in one day.

Evidence: 2 layers: monolith (write) and 3 services (read/async). 7 domain modules with clear names and boundaries.

Reference: monolith/modules/ — one folder per domain

Maintainability

Target: A feature change touches at most 1 module.

Evidence: Verified across 15 feature additions during expansion. Each one touched 1 to 2 modules at most.

Reference: Git history — one commit per feature

Deployability

Target: Full system starts with one command.

Evidence: docker compose up --build works on macOS and Ubuntu 24.04. All 11 containers start in the correct dependency order.

Reference: docker-compose.yml — depends_on with healthcheck conditions

Modularity

Target: Zero cyclic dependencies.

Evidence: ESLint passes on all modules. Each module exposes one index.js.

Reference: monolith/modules/*/index.js

Testability

Target: At least 60% line coverage on core logic.

Evidence: 100% line coverage on bracketUtils.js, resultUtils.js, standingsUtils.js. 12 out of 12 tests pass.

Reference: cd monolith && npm test

Scalability

Target: p95 under 500ms at 50 concurrent users.

Evidence: k6 load test: p95 = 16.27ms, 0% errors, 62 requests per second at peak. All 4 thresholds passed.

Reference: k6 run load-test/load-test.js

Fault tolerance

Target: Monolith stays up if a service crashes.

Evidence: Docker restart:unless-stopped on all services. Health endpoints on all 4 services return status ok.

Reference: GET /health/monolith/, /health/standings/, /health/public/, /health/notification/

Observability

Target: All services expose /metrics. Grafana shows live data.

Evidence: Prometheus scraping 3 services every 15 seconds. 10-panel dashboard loads at <http://localhost:3100>.

Reference: monitoring/prometheus.yml, monitoring/grafana-dashboard.json

Security

Target: JWT on admin routes. Rate limiting on all routes.

Evidence: Nginx returns 429 when the limit is exceeded. All /api/v1/admin/* routes return 401 without a valid token.

Reference: gateway/nginx.conf, monolith/middleware/auth.js

9. Implementation Summary

The base system was built in 7 commits and covers all 10 kata requirements: project scaffold, PostgreSQL schema, monolith with 4 domain modules, standing-service and public-service, React frontend with 7 pages, Nginx gateway, and Jest unit tests.

The expansion added 28 more commits and covers the architectural depth required by the course: RabbitMQ with 3 consumers and a notification-service, Redis cache with event-driven invalidation, health endpoints and rate limiting, API versioning to /api/v1/, Winston structured logging across all 4 services, Prometheus metrics on 3 services, Grafana 10-panel dashboard, k6 load test, SVG bracket tree, Server-Sent Events, landing page with dark mode, multiple tournaments support, player goal tracking and top scorers, auto-generated match reports, bracket seeding, tournament history and archive, team logo upload, live match commentary, and Swagger documentation at /api-docs.

10. Testing and Validation

There are three levels of testing in this project.

Unit tests (automated): Jest tests on the three core utility modules. These are pure functions with no database dependencies, so they run fast and reliably. Running `cd monolith && npm test` gives: 12 out of 12 tests pass, 100% line coverage on `bracketUtils.js`, `resultUtils.js`, and `standingsUtils.js`.

Load test (automated): k6 with 4 stages, ramping up to 50 concurrent virtual users over 3 minutes. Running `k6 run load-test/load-test.js` gives: p95 = 16.27ms, 0% error rate, 62 requests per second at peak, all 4 thresholds passed.

Manual testing: all 16 API endpoints were tested with curl. The full tournament lifecycle was run manually: create tournament, enroll teams, generate bracket, enter results, verify the bracket advances, check standings, view the match report. The SSE update was tested by

opening two browser tabs side by side — when a result was entered in the admin panel, the public bracket updated in the other tab without a page refresh.

One limitation: there are no integration tests that exercise the full HTTP request to database path. The RabbitMQ to SSE chain has no automated test — it was only checked by reading Docker logs. This is the main gap in the current test coverage.

11. Limitations

No integration tests

The only automated tests are unit tests on pure functions. Nothing tests the full request chain from HTTP to database and back. The RabbitMQ event and SSE flow were only tested manually.

Database migrations are manual

When a new column is needed on a running database, it must be added with a manual ALTER TABLE command. There is no migration runner. This happened 6 times during development — once for each new column or table.

Single database with no replication

All services share one PostgreSQL instance. If the database goes down, every service loses its data source at the same time. For a local tournament platform this is acceptable, but it is a single point of failure.

No SSL on the live deployment

The site at tournament.samyland.com runs over HTTP. The admin password is sent in plaintext. This was not fixed because the main domain uses a separate Apache setup and Certbot was not configured for the subdomain.

Admin credentials are hardcoded

The admin username and password are constants in monolith/app.js. There is no user management, no multiple admin accounts, and no way to change the password without editing the code.

Uploaded logos are not persisted across container rebuilds

Team logos are stored inside the monolith container at /app/uploads/. This folder is not a Docker volume. If the container is rebuilt, all logos are lost. This happened once during development when we ran docker compose down -v.

12. Future Improvements

Integration tests

Adding tests that start the full Docker stack and test the complete request-to-database path. Supertest would work for the HTTP layer. A test for the RabbitMQ to SSE chain would also be useful.

Migration runner

Replacing manual ALTER TABLE commands with a tool that applies SQL files automatically on startup. The migration files already exist (001 through 006) — they just need a runner.

SSL

Configuring Certbot for tournament.samyland.com. The Apache VirtualHost on the VPS already supports HTTPS for the main domain.

Persistent uploads

Adding a named Docker volume for /app/uploads/ so logos survive container rebuilds.

Email notifications

notification-service already consumes RabbitMQ events and runs Node.js. Replacing the log statement with an email API call would turn it into a real notification service. The monolith would not need to change at all.

Admin user management

Replacing the hardcoded credentials with a users table in PostgreSQL, and allowing multiple organizer accounts per tournament.

13. How We Used Generative AI

We used Claude Code, Anthropic's CLI coding tool, for all the implementation work. The approach was to fully design the architecture first and then write prompts based on that design. Every prompt referenced specific file paths, endpoint names, database table names, and Docker service names from the blueprint we had built in Tasks 1 and 2. When prompts were precise, the output was usually correct. When prompts were vague, more back-and-forth was needed.

We ran 22 AI coding sessions in total. A few observations:

- Database migrations were the most common correction (6 of 22 sessions). The AI wrote correct SQL migration files but could not run them on the live container. We fixed each one manually using `docker compose exec db psql`.
- Twice the AI added something we had not asked for that turned out to be necessary. For the Grafana setup it added `grafana-dashboards-provider.yaml` — without this file, Grafana silently ignores the dashboard JSON. For Nginx it added resolver `127.0.0.11` — without this, Nginx cannot resolve Docker service names at startup.

- The AI was most consistent when generating the same pattern across multiple files. The Winston logger setup is identical across all 4 services. The Prometheus metricsMiddleware is identical across the 3 services that use it. Writing this by hand would have taken longer and likely introduced inconsistencies.
- The base URL bug we hit on the VPS (api.js was pointing to http://localhost) was not an AI mistake — we had not thought about the difference between development and production when writing the prompt.
- Our role throughout was to design the architecture, write the prompts, review the output, run the tests, fix what was wrong, and push the working code. The AI did not make any architectural decisions.

14. Conclusion

The project ended up covering more than the minimum requirements. The system has a working hybrid architecture, a live deployment on a VPS, automated unit tests, a load test with results, and monitoring through Grafana. All 10 kata requirements are implemented and verified.

The most useful thing we took from the course was the idea that why matters more than how. Having to explain why we used RabbitMQ, why we chose service-based over microservices, and why we used domain partitioning forced us to think more carefully about each decision. Without Tasks 1 and 2, we would have just picked familiar technologies rather than technologies that actually fit the problem.